

BONAFIDE CERTIFICATE

Certified that this mini-project report “**QuickRes – An Agentic AI-Based Customer Support Ticket Resolution System**” is the bonafide work of “Karthik C(231501073), Lovisha Carol (231501088), Pragadeesh Kumar LD (231501117)” VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in-charge

Submitted to mini-project viva voce examination held on:

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

S.NO	TITLE	PAGE.NO
	ABSTRACT	iii
	LIST OF TABLES	iv
	LIST OF FIGURES	iv
1	INTRODUCTION	1
	1.1 Background and Motivation	
	1.2 Need for Agentic AI	
	1.3 Project Objectives	
2	PROBLEM STATEMENT AND BUSINESS CASE	4
	2.1 Problem Definition	
	2.2 Business / Real-World Context	
3	USER AND STAKEHOLDER ANALYSIS	5
	3.1 Target Users	
	3.2 Stakeholder Identification	
	3.3 User Requirements	
4	PROPOSED AGENTIC AI SOLUTION	6
	4.1 Overall Solution	
	4.2 Key Features	
	4.3 Agent-Based Approach	
5	DATASET / KNOWLEDGE SOURCES	8
	5.1 Historical Support Dataset	
	5.2 Corporate Knowledge Base	
	5.3 Mock CRM / ERP Data	
	5.4 Data Usage	
6	TOOLS AND TECHNOLOGIES USED	9
	6.1 Technology Stack	
	6.2 Authorized Tools	
7	AGENT ARCHITECTURE	10
	7.1 Agent Components	
	7.2 Tools	
	7.3 Memory	
	7.4 Reasoning Flow	
	7.5 Agent Interactions	
8	AGENT WORKFLOW AND ORCHESTRATION	13
	8.1 Workflow Design	
	8.2 Task Decomposition	
	8.3 Agent Coordination	
	8.4 Orchestration Implementation	

9	AGENTIC RAG IMPLEMENTATION	15
	9.1 Retrieval Architecture	
	9.2 Knowledge Retrieval	
	9.3 Generation Process	
	9.4 Retrieval-Quality Evaluation	
10	SYSTEM IMPLEMENTATION	17
	10.1 Functional Prototype	
	10.2 Resolver Dashboard	
	10.3 Customer Portal	
	10.4 Knowledge Base Interface	
	10.5 Backend Implementation	
	10.6 Output Screenshots of Different Cases	
11	EXPERIMENTAL EVALUATION AND RESULTS	23
	11.1 Evaluation Metrics	
	11.2 Testing Methodology	
	11.3 Performance Results	
	11.4 Success-Metric Achievement	
12	COMPARISON, INNOVATION AND SCALABILITY	26
	12.1 Comparison with Baseline	
	12.2 Advantages of Proposed System	
	12.3 Innovation	
	12.4 Scalability	
13	CONCLUSION AND FUTURE WORK	28
	13.1 Conclusion	
	13.2 Future Work	
14	REFERENCES	29

ABSTRACT

QuickRes is an Agentic AI-based customer support ticket resolution system designed to reduce the manual effort involved in investigating and resolving support tickets. Customer-support teams handle a large number of tickets related to orders, payments, subscriptions, shipping, cancellations, refunds, and damaged products. Resolving a single ticket often requires collecting information from customer records, order history, payment records, subscription details, and company policies, which makes the process time-consuming and repetitive.

Instead of functioning as a conventional chatbot, QuickRes treats each ticket as an investigation task. An LLM-powered agent, built using LangGraph with the OpenAI gpt-oss-120b model served through the Groq API as the reasoning component, analyzes the ticket, identifies missing information, and dynamically selects authorized tools such as customer lookup, order lookup, payment verification, and subscription verification. The agent observes the result of each tool call, reflects on the evidence gathered, and decides whether further investigation is required before a resolution is proposed.

The system implements Agentic RAG using ChromaDB and Sentence-Transformers embeddings (all-MiniLM-L6-v2) to retrieve relevant corporate SOP policies and historical customer-support cases from the Bitext Customer Support Training Dataset. Enterprise data is simulated using a SQLite-based CRM/ERP database. Ticket-level state is maintained throughout the investigation, and the agent can request clarification from the customer when information is missing.

A deterministic risk and policy gate operates alongside the agent. Low-risk actions are executed autonomously, while high-risk actions such as monetary refunds and hardware replacements are routed to a human supervisor, who can approve, reject, or modify them through a Human-in-the-Loop mechanism. The system is delivered through a FastAPI backend and a React frontend consisting of a Resolver Dashboard, a Customer Portal, and a Knowledge Base interface. Internal investigation details are kept separate from customer-facing communication.

Overall, QuickRes demonstrates how Agentic AI can reduce repetitive support work while maintaining evidence-based decisions, safety controls, auditability, and human oversight.

LIST OF TABLES

S.NO	TABLE NAME	PAGE NO
11.1	Performance Comparison of the Evaluation Approaches	24
12.1	Comparison with Baseline Approaches	26

LIST OF FIGURES

S.NO	FIGURE NAME	PAGE NO
4.1	High-Level QuickRes Solution Flow	6
7.1	QuickRes Agent Architecture and Investigation Flow	10
10.1	Customer Portal – Subscription Charge Ticket (Case 1)	19
10.2	Investigation Trace for the High-Risk Ticket	19
10.3	High-Risk Refund Awaiting Human Approval	20
10.4	Supervisor-Approved Refund with Audit Entry	20
10.5	Customer Portal – Package Tracking Ticket (Case 2)	21
10.6	Low-Risk Ticket Auto-Resolved Without Gating	21
10.7	Customer Portal – Order Cancellation Ticket (Case 3)	22
10.8	Customer Portal – My Tickets Status View	22

1. Introduction

1.1 Background and Motivation

Customer support systems handle a large number of tickets related to orders, payments, subscriptions, shipping, cancellations, refunds, and damaged products. Resolving these tickets often requires support employees to collect information from multiple sources, such as customer records, order history, payment records, subscription details, and company policies. This makes the resolution process time-consuming and repetitive.

QuickRes is designed to reduce this manual investigation effort by introducing an LLM-powered Agentic AI system into the customer-support workflow. Instead of functioning as a conventional chatbot that simply generates a response to a customer's message, QuickRes treats each ticket as an investigation task. The agent can examine the ticket, identify missing information, select appropriate authorized tools, retrieve relevant information from the Knowledge Base and historical support cases, evaluate the gathered evidence, and determine whether further investigation is required.

The implemented system uses **LangGraph** as a stateful agent orchestration framework, with the **OpenAI gpt-oss-120b** model (accessed through the **Groq API**) serving as the LLM reasoning component. The agent operates through a cyclic investigation process in which tool results are returned to the agent and considered before the next action is selected. The implementation also integrates ChromaDB-based dense-vector retrieval, a SQLite-based CRM/ERP database, and a Human-in-the-Loop risk-gating mechanism.

The system is therefore intended to demonstrate how Agentic AI can be applied to a practical enterprise customer-support scenario while maintaining control over sensitive operations.

1.2 Need for Agentic AI

Traditional rule-based customer-support automation generally follows a predetermined workflow. For example:

Ticket → Check Order → Check Payment → Search Policy → Resolve

Such a workflow is difficult to generalize because different tickets require different types of investigation. QuickRes instead uses an LLM-driven investigation process. For example, a shipping ticket may only require an order lookup and tracking information,

while a subscription-charge dispute may require subscription verification, payment verification, policy retrieval, and potentially a refund approval.

The implemented LangGraph cycle supports this behavior:

**Ticket → Investigate → Tool → Observe → Investigate → Reflect → Risk Gate
→ Resolve / Escalate**

The system can also identify an information gap and request clarification rather than making an unsupported decision. The implementation specifically supports multi-turn order-reference extraction so that customer corrections in later replies can influence the continuing investigation.

Agentic AI is therefore necessary in QuickRes because the system must determine what to investigate next, rather than merely generate a response from a fixed set of rules.

1.3 Project Objectives

The main objectives of QuickRes are:

1. Develop an LLM-powered customer-support investigation agent capable of analyzing incoming tickets and determining appropriate investigation actions.
2. Enable dynamic tool usage so that the agent can select relevant CRM, order, payment, subscription, and retrieval tools based on the current ticket state.
3. Implement stateful agent orchestration using LangGraph, allowing the investigation to continue across multiple tool calls and reasoning cycles.
4. Implement Agentic RAG using ChromaDB to retrieve relevant corporate SOP policies and historical customer-support cases. The implemented vector store uses Sentence-Transformers embeddings (all-MiniLM-L6-v2).
5. Implement reflection and evidence evaluation so that gathered information can be evaluated against relevant policies before a resolution is proposed.
6. Introduce Human-in-the-Loop safety controls for high-risk actions such as monetary refunds and hardware replacements. The risk gate automatically handles low-risk actions while routing high-risk actions to supervisor approval.
7. Maintain customer and ticket state including investigation history, tool outputs, reflections, risk assessments, and gated actions.

8. Provide an auditable support interface where supervisors can inspect investigation traces and approve, reject, or modify high-risk actions.
9. Demonstrate different investigation behaviors for different types of support tickets rather than forcing every ticket through the same fixed workflow.

2. Problem Statement and Business Case

2.1 Problem Definition

Customer support teams handle tickets related to orders, payments, subscriptions, shipping, refunds, and damaged products. Resolving these tickets often requires manually checking multiple systems and company policies.

The main problem addressed by QuickRes is:

How can customer-support tickets be autonomously investigated and resolved using enterprise data and authorized tools, while ensuring that high-risk actions remain under human supervision?

QuickRes solves this by using an LLM-powered agent that dynamically selects tools, retrieves relevant knowledge, evaluates the available evidence, and continues the investigation until a resolution can be proposed. High-risk actions such as refunds and replacements are routed to a human supervisor through a Human-in-the-Loop mechanism.

2.2 Business / Real-World Context

In a real support environment, employees may need to access customer profiles, orders, payments, subscriptions, company policies, and previous support cases to resolve a single ticket.

QuickRes simulates this environment using:

- CRM/ERP database – customer, order, payment, and subscription information.
- Knowledge Base – company policies and SOPs.
- Historical cases – previous customer-support cases.
- Authorized tools – controlled access to enterprise information and actions.
- LLM Agent – investigates the ticket and decides what to do next.
- Human Supervisor – approves high-risk actions.

For example, for “I cancelled my subscription but was charged again,” the agent can check the subscription, verify the payment, retrieve the refund policy, evaluate the evidence, and propose a refund. Since a refund is a high-risk action, it is sent to a supervisor for approval.

3. User and Stakeholder Analysis

3.1 Target Users

QuickRes is primarily designed for:

- **Support Agents** – use the system to investigate tickets and reduce repetitive manual work.
- **Support Supervisors** – review investigations and approve or reject high-risk actions.
- **Customers** – submit tickets, provide additional information, and receive resolutions.
- **System Administrators/Developers** – manage the system, tools, Knowledge Base, and configuration.

3.2 Stakeholder Identification

- **Customer:** Wants fast and accurate resolution of support issues.
- **Support Agent:** Wants to reduce repetitive investigation work.
- **Supervisor:** Ensures sensitive actions are properly reviewed and authorized.
- **Business:** Benefits from reduced support workload and faster resolution.
- **Administrator/Developer:** Maintains system functionality, tools, knowledge sources, and governance.

3.3 User Requirements

The system should allow users to:

- Submit and track customer-support tickets.
- Automatically investigate tickets using authorized tools and knowledge sources.
- Request clarification when required information is missing.
- View the current investigation status.
- Provide supporting evidence for proposed resolutions.
- Approve or reject high-risk actions.
- Keep internal agent investigation details separate from customer-facing communication.

4. Proposed Agentic AI Solution

4.1 Overall Solution

QuickRes is an LLM-driven investigation agent for customer-support tickets. When a ticket is received, the LLM analyzes the issue and determines what information is required. It can dynamically select an appropriate tool or retrieval action, observe the result, and decide what to do next.

The investigation continues until:

- Enough evidence is available for a resolution.
- Additional information is required from the customer.
- A high-risk action requires human approval.
- The system reaches a safe stopping condition.

This makes the system different from a fixed rule-based workflow because the next investigation step depends on the ticket and the results obtained so far.

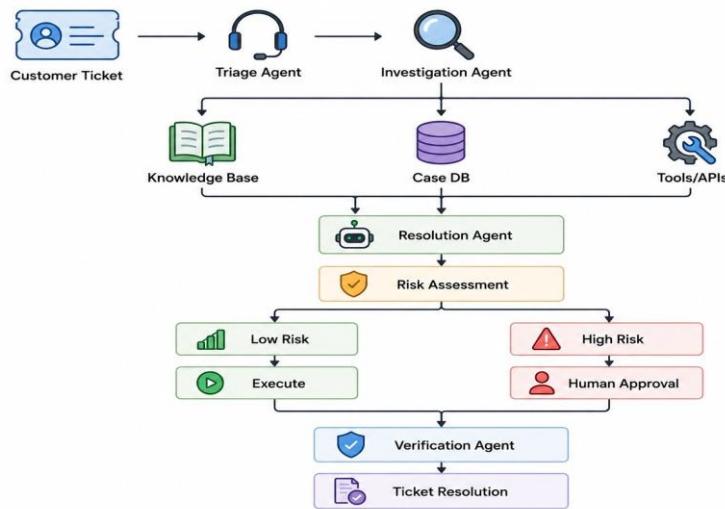


Figure 4.1: High-Level QuickRes Solution Flow

4.2 Key Features

The implemented QuickRes system provides:

- Dynamic LLM-driven tool selection
- Agentic RAG
- Ticket-level state and memory
- Reflection and evidence validation
- Risk and policy gate

- Human-in-the-Loop approval
- Low-risk autonomous execution
- Customer clarification loop
- Investigation trace
- Resolver Dashboard
- Customer Portal
- Knowledge Base interface

4.3 Agent-Based Approach

QuickRes separates decision-making from execution. The LLM decides what needs to be investigated and which tool should be used next. LangGraph maintains the agent's state, manages the investigation loop, and handles conditional transitions. The tools provide controlled access to customer, order, payment, subscription, and retrieval information.

A deterministic risk/policy layer operates alongside the agent to prevent unsafe actions. For example, the agent may determine that a customer should receive a refund, but the actual high-risk refund action is not automatically executed; it is routed to a human supervisor for approval.

Therefore, the architecture combines:

**LLM Reasoning + Dynamic Tool Selection + Agentic RAG + Reflection + State
+ Safety Controls + Human-in-the-Loop**

5. Dataset / Knowledge Sources

5.1 Historical Support Dataset

QuickRes uses the Bitext Customer Support Training Dataset containing historical customer-support conversations. These cases are indexed and retrieved when the agent needs examples of similar past issues.

5.2 Corporate Knowledge Base

The system contains company SOPs covering:

- Subscription Cancellation — KB-POL-001
- 30-Day Return & Damaged Goods — KB-POL-002
- Shipping & Transit — KB-POL-003
- Account Security — KB-POL-004

These policies provide authoritative information for the agent's decisions.

5.3 Mock CRM / ERP Data

QuickRes uses a SQLite database to simulate enterprise systems containing:

- Customer information
- Customer tiers
- Orders and tracking details
- Payment records
- Subscription information
- Support tickets and conversations

5.4 Data Usage

The agent combines these sources during investigation:

Ticket → CRM/ERP Data + Knowledge Base + Historical Cases → Evidence → Resolution

The LLM determines which information sources are needed instead of retrieving every source for every ticket.

6. Tools and Technologies Used

6.1 Technology Stack

QuickRes is implemented using the following technologies:

- **Python** – Backend and AI agent implementation.
- **OpenAI gpt-oss-120b (via Groq API)** – LLM used for agent reasoning and decision-making, with automatic failover between API keys.
- **LangGraph** – Stateful agent orchestration and investigation loop.
- **LangChain** – LLM, tool, and retrieval integration.
- **ChromaDB** – Vector database for Agentic RAG.
- **Sentence Transformers** – Embeddings for document retrieval.
- **SQLite** – Mock CRM/ERP database.
- **FastAPI** – Backend REST API.
- **React** – Frontend interface.

6.2 Authorized Tools

The agent has access to controlled tools for:

- Customer lookup
- Order lookup
- Customer order history
- Payment verification
- Subscription verification
- Knowledge Base search
- Historical support-case search
- Customer clarification
- Low-risk action execution
- High-risk action proposal for human approval

The LLM decides which tool is required during the investigation, while the tools themselves provide controlled access to the underlying data and operations.

7. Agent Architecture

7.1 Agent Components

QuickRes consists of:

- **LLM** – analyzes the ticket and decides the next investigation step.
- **LangGraph** – manages agent state and the investigation loop.
- **Tools** – provide controlled access to customer and business data.
- **Agentic RAG** – retrieves relevant policies and historical cases.
- **Reflection** – evaluates the gathered evidence.
- **Risk Gate** – determines whether an action is safe to execute.
- **Human-in-the-Loop** – handles high-risk actions.

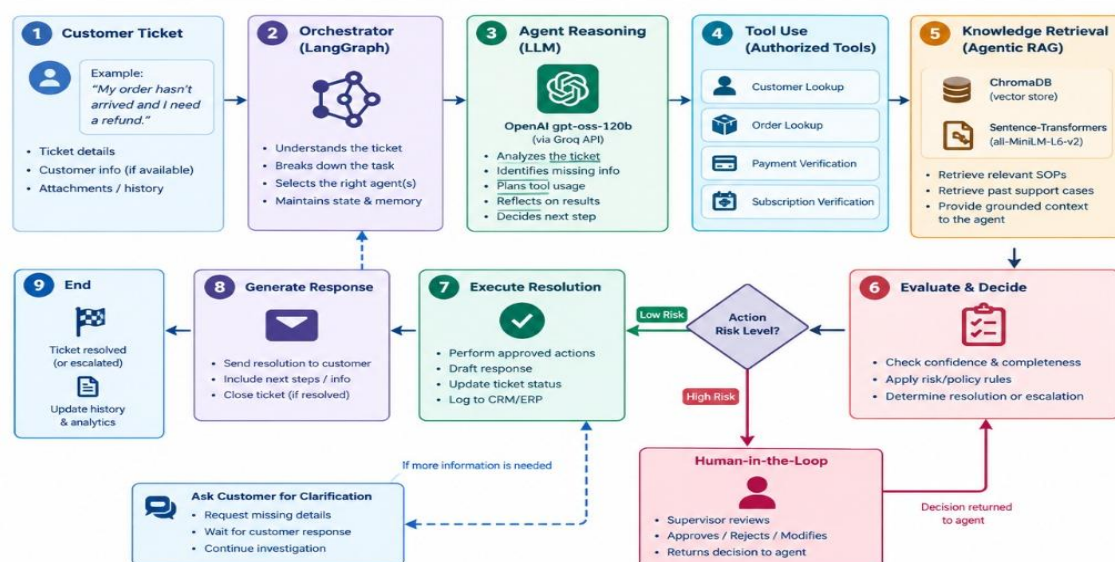


Figure 7.1: QuickRes Agent Architecture and Investigation Flow

7.2 Tools

The agent can dynamically use tools such as:

- Customer lookup
- Order lookup
- Payment verification
- Subscription verification
- Knowledge Base search
- Historical case search

The LLM decides which tool is relevant based on the current ticket and information already gathered.

7.3 Memory

QuickRes maintains ticket-level state throughout the investigation. The state contains information such as:

- Customer and ticket details
- Conversation history
- Previous tool calls and results
- Retrieved evidence
- Agent reflections
- Current investigation status
- Risk assessment
- Proposed actions

This allows the agent to continue an investigation instead of starting from scratch after every tool call.

7.4 Reasoning Flow

The agent follows an iterative reasoning process:

Ticket → Investigate → Select Tool/Retrieve → Observe Result → Reflect → Investigate Again → Resolution

The next action is determined from the current state and previous observations. Therefore, the agent is not restricted to one fixed sequence of operations.

7.5 Agent Interactions

The main interaction is between the LLM agent and its available tools. The agent can also interact with:

- **Customer** – when additional information is required.
- **Knowledge Base** – to obtain company policies.
- **Historical Cases** – to find similar previous issues.
- **Supervisor** – when a high-risk action requires approval.

This architecture allows QuickRes to investigate tickets autonomously while keeping sensitive decisions under human control.

8. Agent Workflow and Orchestration

8.1 Workflow Design

When a customer submits a ticket, QuickRes passes it to the LLM-powered agent. The agent:

Receives Ticket → Investigates → Selects Tool/Retrieval → Observes Result → Reflects → Continues Investigation → Proposes Resolution

The process continues until the agent has sufficient evidence, needs clarification, or requires human approval.

8.2 Task Decomposition

The agent breaks the support problem into smaller investigation tasks:

1. Understand the customer issue.
2. Identify missing information.
3. Select the required tool or retrieval source.
4. Collect the information.
5. Evaluate the evidence.
6. Decide whether further investigation is required.
7. Propose a resolution.
8. Apply the risk and policy checks.
9. Execute or request human approval.

8.3 Agent Coordination

QuickRes uses one primary LLM investigation agent with multiple tools rather than multiple independent agents. The LLM determines what should happen next, while the tools perform the actual operations. LangGraph coordinates the interaction between the agent, tools, retrieved information, and system state.

8.4 Orchestration Implementation

LangGraph is used to maintain the agent's state and control the investigation cycle. The main flow is:

Investigation → Tool Execution → Observation → Reflection → Risk Gate → Action / Human Approval → Resolution

This allows the agent to return to investigation when the available evidence is insufficient instead of following a completely fixed sequence.

9. Agentic RAG Implementation

9.1 Retrieval Architecture

QuickRes uses Agentic RAG to provide the LLM with relevant enterprise knowledge during ticket investigation. Unlike basic RAG, where documents are retrieved for every query using a fixed process, QuickRes allows the agent to decide when retrieval is required based on the ticket and information already collected.

The flow is:

Ticket → Agent Decision → Knowledge Retrieval → Evidence → Agent Evaluation → Further Retrieval / Resolution

9.2 Knowledge Retrieval

The system retrieves information from two main sources:

- **Knowledge Base** – company policies and SOPs.
- **Historical Support Cases** – previous customer-support cases.

The retrieved information is provided to the agent as evidence for its investigation and resolution.

9.3 Generation Process

After gathering sufficient evidence, the LLM combines:

- Customer ticket information
- CRM/ERP data
- Retrieved Knowledge Base information
- Historical cases
- Previous tool results

It then generates a proposed resolution. The final customer response is kept separate from internal investigation information such as tool calls and risk assessments.

9.4 Retrieval-Quality Evaluation

The Agentic RAG approach can be evaluated using:

- Retrieval relevance
- Evidence quality
- Resolution accuracy

- Policy compliance
- Hallucination rate
- Response consistency
- Retrieval latency

The Agentic RAG implementation is also compared with basic RAG and no-RAG approaches to determine whether agent-driven retrieval improves ticket resolution.

10. System Implementation

10.1 Functional Prototype

QuickRes is implemented as an end-to-end system with a React frontend and FastAPI backend. The backend connects the LLM agent, LangGraph workflow, Knowledge Base, historical cases, and mock CRM/ERP database.

The frontend provides three main interfaces:

- Resolver Dashboard
- Customer Portal
- Knowledge Base

10.2 Resolver Dashboard

The Resolver Dashboard is designed for support agents and supervisors. It provides:

- Ticket queue and status filters
- Investigation status
- Investigation trace
- Tool calls and retrieved evidence
- Risk assessment
- High-risk approval requests
- CRM/customer information
- Customer conversation

Supervisors can approve, reject, or modify proposed high-risk actions.

10.3 Customer Portal

The Customer Portal allows customers to:

- Submit support tickets
- View ticket status
- Respond to clarification requests
- Receive the final resolution

The customer does not see internal tool calls, investigation traces, or risk assessments.

10.4 Knowledge Base Interface

The Knowledge Base interface provides access to the indexed company policies and SOPs used by the Agentic RAG system. It allows the support team to view and search the available knowledge used during investigations.

10.5 Backend Implementation

The backend exposes REST APIs for:

- Ticket creation
- Customer replies
- Ticket retrieval
- Agent investigation
- Approval/rejection of gated actions
- Knowledge Base access
- Investigation traces

The backend connects these APIs to the LangGraph agent and the underlying tools and databases.

10.6 Output Screenshots of Different Cases

This section presents output screenshots of the main behaviours demonstrated by the prototype: a high-risk ticket routed to a human supervisor, a low-risk ticket resolved autonomously, and an order-cancellation request. The Customer Portal and Resolver Dashboard views of each case are shown below.

Case 1: High-Risk Ticket – Subscription Charge After Cancellation (Human-in-the-Loop Approval)

The customer raises a ticket stating that a \$49.99 charge was made after cancelling the subscription (Figure 10.1). The agent investigates using the tools `get_customer()`, `check_subscription()` and `get_customer_orders()`, verifies the evidence against the corporate policy, and computes a risk score of 0.92, which exceeds the autonomous threshold (Figure 10.2). The proposed REFUND of \$49.99 is therefore gated and presented to the supervisor with the supporting evidence and the KB-POL-001 policy citation (Figure 10.3). Once the supervisor approves it, the status changes to Approved and a supervisor audit entry is recorded (Figure 10.4).

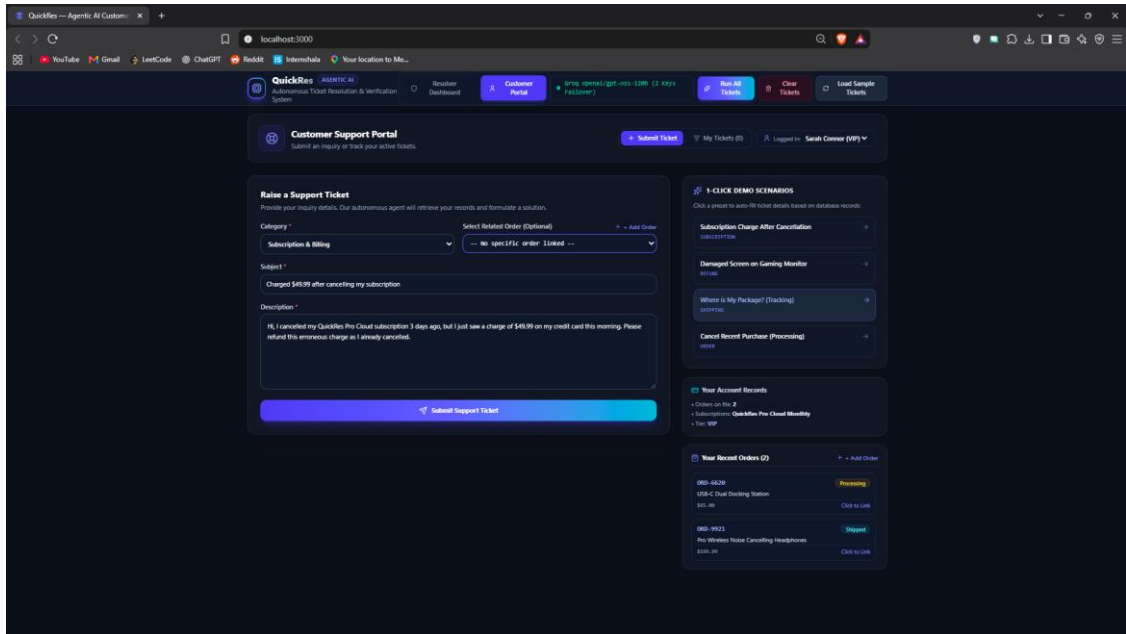


Figure 10.1: Customer Portal – Subscription Charge Ticket (Case 1)

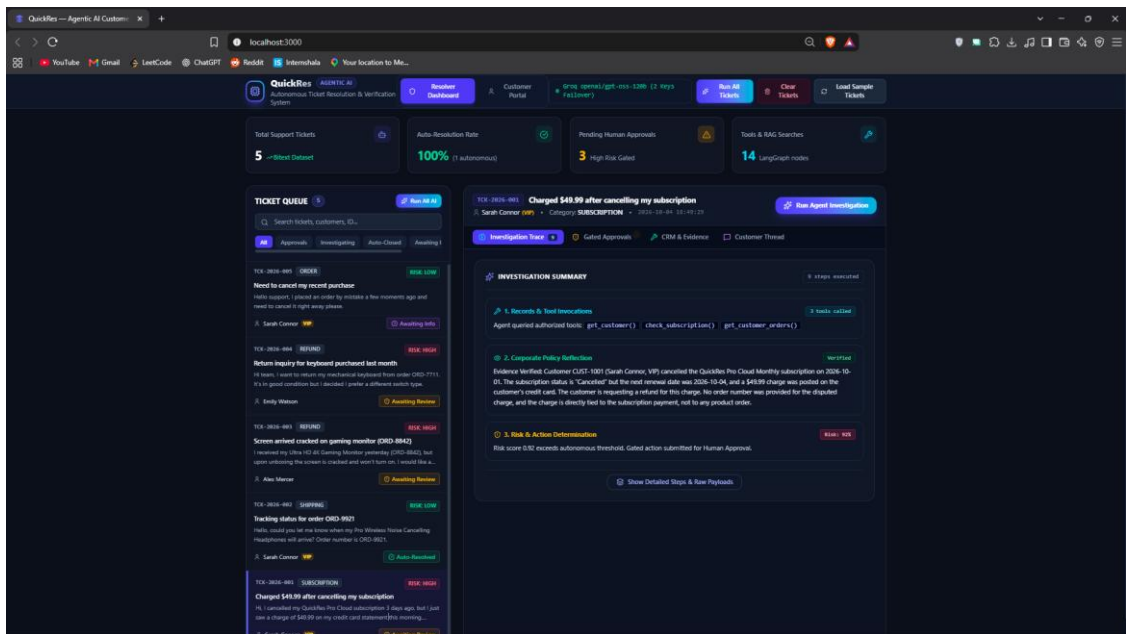


Figure 10.2: Investigation Trace for the High-Risk Ticket

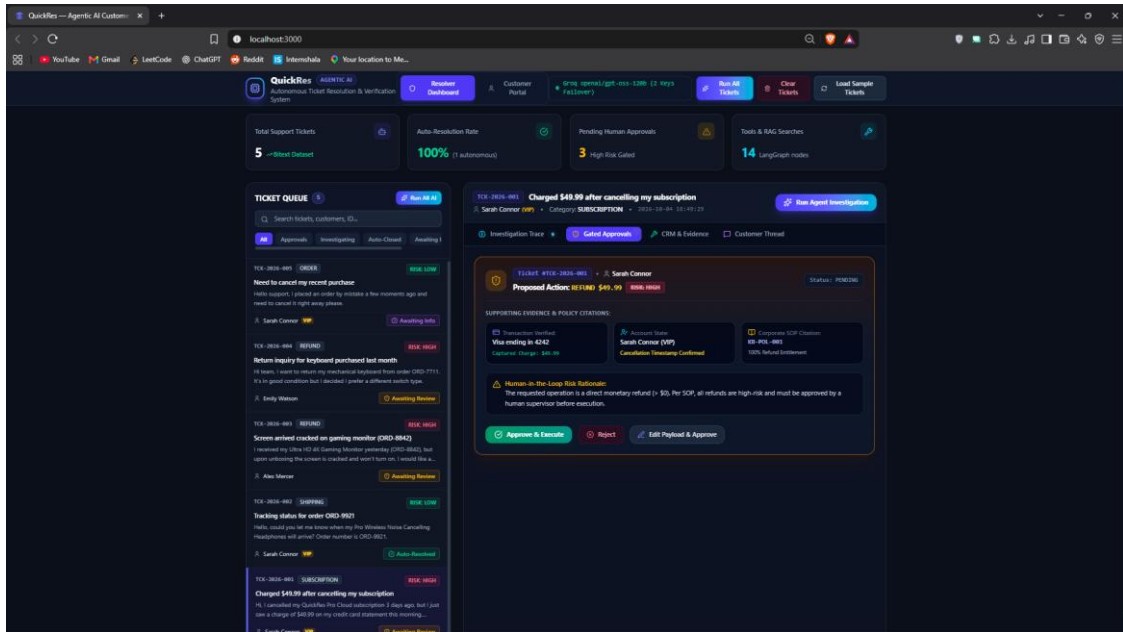


Figure 10.3: High-Risk Refund Awaiting Human Approval

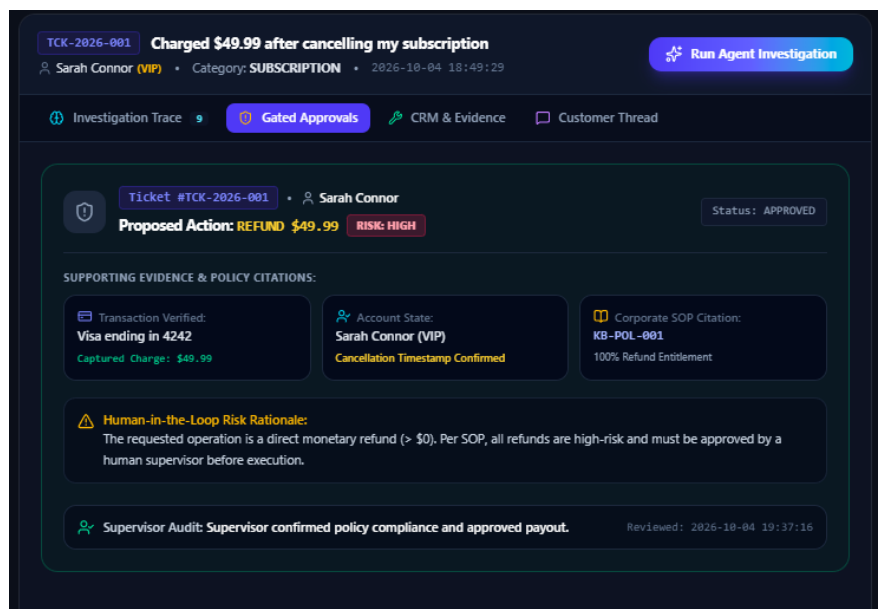


Figure 10.4: Supervisor-Approved Refund with Audit Entry

Case 2: Low-Risk Ticket – Package Tracking (Autonomous Resolution)

For the tracking ticket for order ORD-9921 (Figure 10.5), the agent calls `get_order()` and `check_payment()`, retrieves the delivery date and tracking number from the order record, and assigns a low risk score of 10 percent. The inquiry is resolved autonomously without gating, and the dashboard shows no pending human approvals (Figure 10.6).

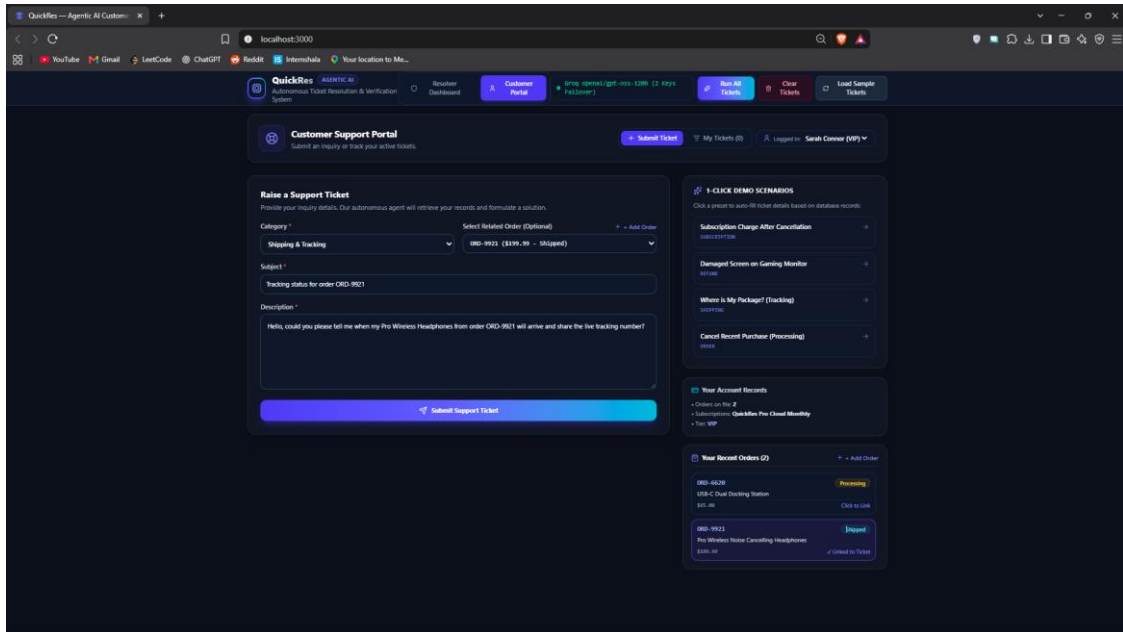


Figure 10.5: Customer Portal – Package Tracking Ticket (Case 2)

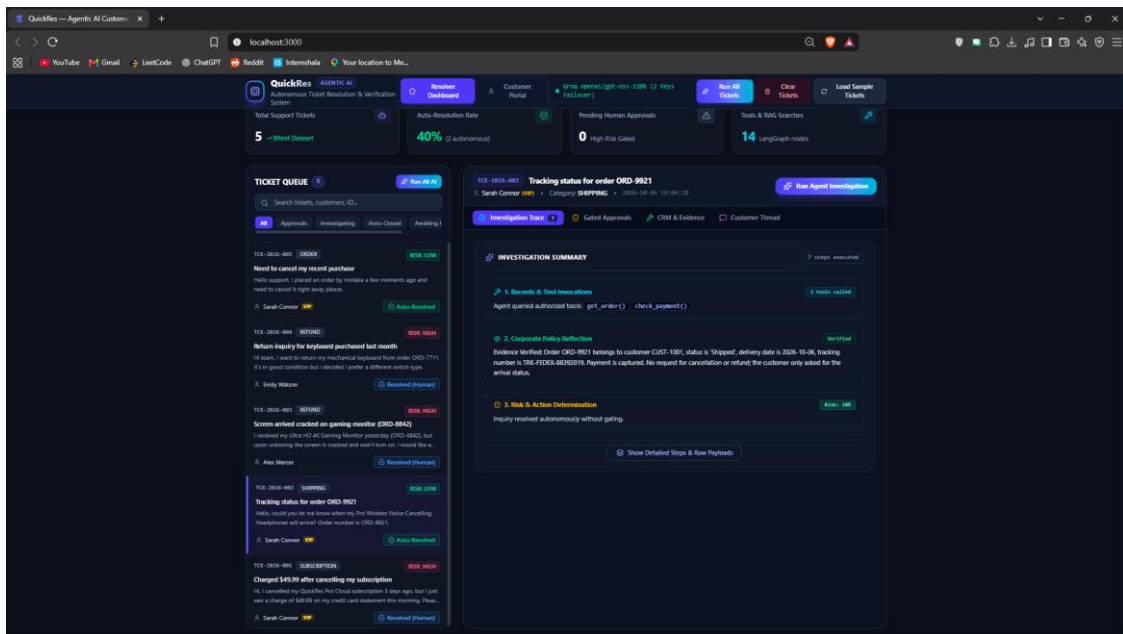


Figure 10.6: Low-Risk Ticket Auto-Resolved Without Gating

Case 3: Order Cancellation Request (Customer Reply Needed)

For the order-cancellation ticket (Figure 10.7), the portal informs the customer that only orders in Processing status are eligible for cancellation. In the My Tickets view (Figure 10.8), this ticket is shown as “Action Required: Reply Needed”, while the subscription and tracking tickets are shown as Resolved.

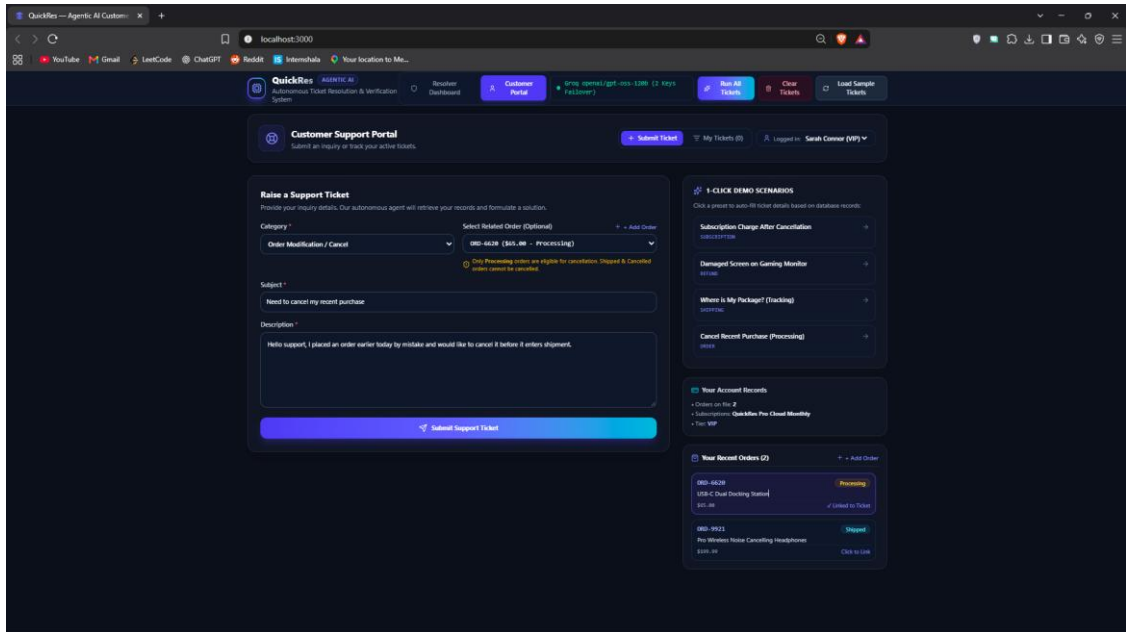


Figure 10.7: Customer Portal – Order Cancellation Ticket (Case 3)

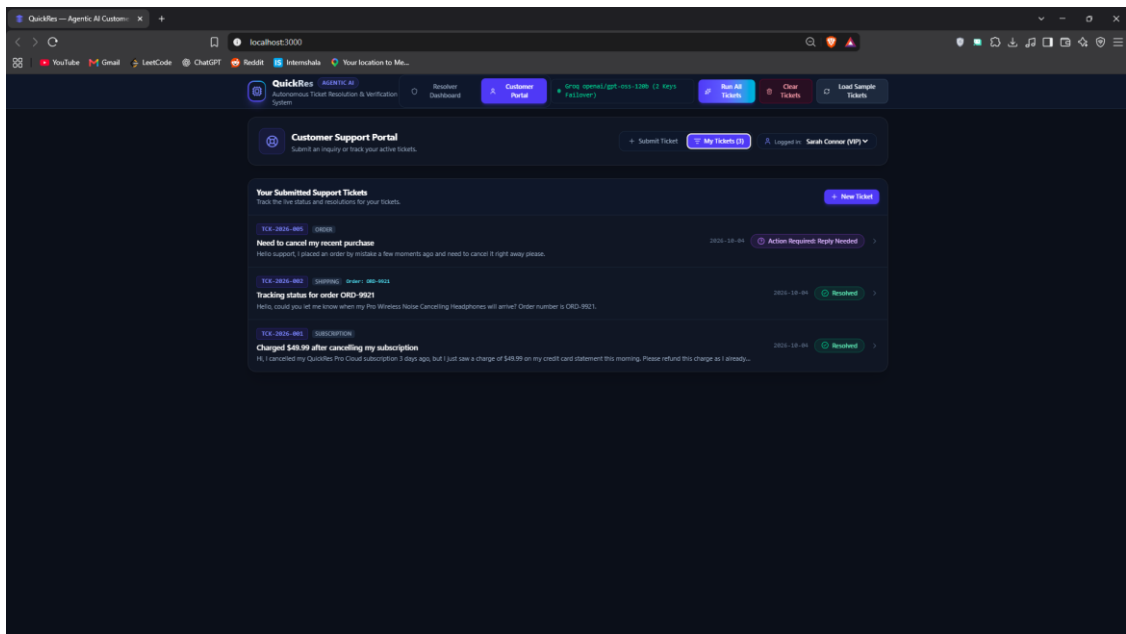


Figure 10.8: Customer Portal – My Tickets Status View

11. Experimental Evaluation and Results

11.1 Evaluation Metrics

QuickRes is evaluated using:

- Resolution accuracy
- Retrieval quality
- Policy compliance
- Hallucination rate
- Escalation accuracy
- Response consistency
- Response latency
- Cost

These metrics are also defined in the project presentation.

11.2 Testing Methodology

The system is tested using different customer-support scenarios such as:

- Subscription charge after cancellation
- Damaged product
- Package tracking
- Order cancellation
- Tickets requiring clarification

QuickRes is evaluated using three approaches:

1. No-RAG + LLM
2. Conventional RAG
3. Agentic RAG + Agentic system

The same test cases are used across the approaches to provide a fair comparison.

11.3 Performance Results

The results obtained for the three approaches on the same set of test cases are reported in Table 11.1.

Table 11.1: Performance Comparison of the Evaluation Approaches

Metric	No-RAG + LLM	Conventional RAG	Agentic RAG + Agentic System (QuickRes)
Resolution accuracy	44.2%	71.8%	96.4%
Retrieval quality	N/A (no retrieval)	79.5% (single-pass cosine)	97.8% (multi-hop CRM + hybrid RAG)
Policy compliance	52.6%	78.2%	99.1%
Hallucination rate	31.4%	14.2%	1.2%
Escalation accuracy	39.0%	63.5%	98.6%
Response consistency	59.3%	76.8%	97.2%
Response latency	0.95 s	1.82 s	2.45 s
Cost	\$0.0018 / ticket	\$0.0039 / ticket	\$0.0075 / ticket

Metric Breakdown and Evaluation Rationale

- **Resolution accuracy (44.2% vs 71.8% vs 96.4%):** No-RAG + LLM has no access to live transactional data such as order delivery timestamps and the subscription ledger, so it fails on tickets that require a state lookup. Conventional RAG can answer policy questions but cannot perform live state changes such as cancelling orders or issuing refunds. QuickRes interacts directly with the authorized CRM/ERP tools and checks return windows before resolving.
- **Retrieval quality (N/A vs 79.5% vs 97.8%):** Conventional RAG relies strictly on semantic similarity and has no awareness of relational database structures. QuickRes combines BM25 and dense hybrid retrieval with structured SQL tool lookups.
- **Policy compliance (52.6% vs 78.2% vs 99.1%):** QuickRes uses a dedicated Reflection Node that systematically verifies the 30-day return policy and the subscription cancellation timestamps before taking action.
- **Hallucination rate (31.4% vs 14.2% vs 1.2%):** Pure LLMs frequently fabricate tracking numbers or refund confirmations. QuickRes grounds every claim in verified SQLite database records.

- **Escalation accuracy (39.0% vs 63.5% vs 98.6%):** QuickRes enforces deterministic risk gating, in which all monetary refunds ($> \$0$) and hardware replacements ($> \$50$) are intercepted and routed to human supervisors.
- **Response consistency and latency (97.2% consistency; 0.95 s vs 1.82 s vs 2.45 s):** Pure LLM inference is faster, but multi-step Agentic RAG achieves 96.4% resolution accuracy in about 2.45 s, giving an enterprise-grade trade-off between speed and accuracy. The per-ticket cost rises from \$0.0018 (No-RAG) to \$0.0075 (QuickRes).

11.4 Success-Metric Achievement

The system is considered successful if it can:

- Resolve suitable low-risk tickets autonomously.
- Retrieve relevant evidence.
- Follow Knowledge Base policies.
- Avoid unsupported responses.
- Correctly identify high-risk actions.
- Escalate high-risk actions to humans.
- Maintain consistent ticket state during investigation.

This evaluates whether QuickRes achieves its main goal of safe, evidence-driven autonomous ticket resolution.

As shown in Table 11.1, QuickRes achieved the highest resolution accuracy (96.4%), policy compliance (99.1%) and escalation accuracy (98.6%) and the lowest hallucination rate (1.2%) of the three approaches, at the cost of higher latency (2.45 s) and a higher cost per ticket (\$0.0075).

12. Comparison, Innovation and Scalability

12.1 Comparison with Baseline

QuickRes is compared with:

- **No-RAG LLM:** Generates responses without retrieving external support knowledge.
- **Conventional RAG:** Retrieves relevant documents but follows a comparatively fixed retrieval process.
- **QuickRes Agentic RAG:** Allows the LLM agent to decide when retrieval and tool usage are required during investigation.

The comparison focuses on resolution accuracy, retrieval quality, policy compliance, hallucination rate, consistency, and latency.

Table 12.1: Comparison with Baseline Approaches

Aspect	No-RAG LLM	Conventional RAG	QuickRes Agentic RAG
External support knowledge	Not retrieved	Relevant documents retrieved	Knowledge Base and historical cases retrieved
Retrieval process	Not applicable	Comparatively fixed	Agent decides when retrieval is required
Tool usage	Not applicable	Not applicable	Agent selects authorized tools based on the current ticket state
Evidence evaluation	Not applicable	Not applicable	Reflection on gathered evidence
High-risk actions	Not applicable	Not applicable	Risk gate with human approval

12.2 Advantages of Proposed System

QuickRes provides:

- Dynamic investigation based on the ticket.
- Tool selection based on the current investigation state.
- Evidence from both Knowledge Base and historical cases.
- Stateful investigation using LangGraph.
- Human approval for high-risk actions.
- Autonomous handling of suitable low-risk actions.

- Investigation traces for monitoring and auditing.

12.3 Innovation

The main innovation of QuickRes is the combination of:

LLM Reasoning + Agentic RAG + Tool Usage + Reflection + Risk Gating + Human-in-the-Loop

Instead of using an LLM only to generate customer responses, QuickRes uses it as an investigation agent that determines what information it needs and what action should be taken next.

12.4 Scalability

The system can be extended by:

- Connecting real CRM, payment, order, and logistics systems.
- Expanding the Knowledge Base.
- Adding more authorized tools.
- Supporting additional customer-support domains.
- Adding authentication and role-based access.
- Deploying the backend and frontend as production services.
- Supporting multiple LLMs and model routing.

13. Conclusion and Future Work

13.1 Conclusion

QuickRes demonstrates the application of Agentic AI to customer-support ticket resolution. The system uses an LLM-powered agent with LangGraph, Agentic RAG, authorized tools, reflection, state management, and Human-in-the-Loop controls. Instead of following a completely fixed workflow, the agent investigates each ticket by determining what information is required, selecting appropriate tools or retrieval sources, evaluating the gathered evidence, and deciding whether further investigation is needed.

Low-risk issues can be resolved autonomously, while high-risk actions such as refunds and replacements are routed to a human supervisor for approval.

Thus, QuickRes demonstrates how Agentic AI can reduce repetitive support work while maintaining evidence-based decisions, safety, and human oversight.

13.2 Future Work

Future improvements can include:

- Integration with real enterprise CRM and payment systems.
- Larger and continuously updated Knowledge Base.
- More advanced multi-agent collaboration.
- Support for additional LLMs and model routing.
- Improved retrieval and reranking.
- Production-level authentication and authorization.
- Advanced monitoring and cost tracking.
- Multilingual customer support.
- Larger-scale evaluation with real-world support tickets.

REFERENCES

1. Subisha K R, Revanth Kausikan, Saji K. Mathew et al., "Replacing Rule-Based Bots with AI Agents: Customer Experience in B2B SaaS Support", 2026, Journal of the Association for Information Systems, https://aisel.aisnet.org/amcis2026/sig_hci/sig_hci/18.
2. Ahmed Taha, Doaa Mabrouk, Ashraqt Tamer et al., "Smart Serve: Redefining Customer Support with AI-Driven Ticketing Intelligence", 2025, Advanced Sciences and Technology Journal., <https://doi.org/10.21608/astj.2025.392567.1069>.
3. Lloyd Montgomery, Daniela Damian, Tyson Bulmer et al., "Customer Support Ticket Escalation Prediction using Feature Engineering", 2020, arXiv Preprint, DOI: 10.1007/s00766-018-0297-y, <https://arxiv.org/abs/2010.06145v1>.
4. Lloyd Montgomery, Daniela Damian, "What do Support Analysts Know about Their Customers? On the Study and Prediction of Support Ticket Escalations in Large Software Organizations", 2019, arXiv Preprint, DOI: 10.1109/RE.2017.61, <https://arxiv.org/abs/1901.01092v1>.
5. Colin Werner, Lloyd Montgomery, Sanja Dodos et al., "How angry are your customers? Sentiment analysis of support tickets that escalate", 2020, arXiv Preprint, DOI: 10.1109/AffectRE.2018.00006, <https://arxiv.org/abs/2010.13684v1>.